shmap-rs: an exact, parallel and memory-lean Rust port of shmap

Dobromir Panchev Pesho Ivanov

Abstract

Summary: *map-shmap*, by Ivanov and Medvedev [1], maps long reads by comparing FracMinHash sketches under a seed heuristic that prunes candidate regions before refining them. Their reference implementation (`https://github.com/pesho-ivanov/shmap`) is single-threaded and sizes its bucket accumulator against the *reference*, putting whole-genome mapping outside ordinary memory budgets. We present **shmap-rs**, a Rust port that leaves the algorithm untouched — output is byte-identical — and rebuilds the data structures and the parallel decomposition around it. Against the tuned C++ it is 2.14–2.97× faster single-threaded, uses 7.0–7.3× less peak memory and makes that footprint a property of the query rather than of the genome, and adds thread parallelism the reference lacks, reaching 16.6× on 64 cores. Every claim is measured on two architectures whose runs agree on every algorithmic counter.

Availability: original `https://github.com/pesho-ivanov/shmap`; this port `https://github.com/d0bromir/shmap-rs`, MPL-2.0, version 1.4.1, with the suite, result sets and the generator of every figure here.

Contact: ORCID Panchev 0009-0000-2636-2593; Ivanov 0000-0002-8119-3849.

1 Introduction

Sketch-based mappers compare a read to a genome in the space of a few thousand hashes rather than a few billion bases [2]. *map-shmap* [1] sketches read and reference by FracMinHash, accumulates seed matches into overlapping buckets, prunes them with an admissible seed heuristic, and refines the survivors with a sliding window.

Everything mapping-relevant here is Ivanov's. The algorithm, the seed heuristic and its admissibility argument, the metrics, the parameterisation and the C++ implementation we measure against are all his; this paper contributes no algorithmic idea and reports no accuracy improvement, because there is none to report. What it contributes is engineering: the same algorithm's cost is dominated by dependent memory references rather than arithmetic, and that was worth re-engineering. No change may alter a single output record, so every number below compares two implementations of one algorithm — his.

2 Implementation

Accumulator sized by the read, not the reference. The reference allocates a dense array indexed by reference bucket, so its footprint is a property of the genome and is paid on every run. A read can only match buckets within its own span, so shmap-rs sizes the array by the read's half-length and falls back to a sparse map. Almost the whole memory result, and it removed a contention bottleneck that had made multithreading nearly useless.

Memoised refinement. The second-best-mapping search repeats scoring the best-mapping search has done; shmap-rs replays those scores, removing roughly two calls in five into refinement, for the metrics where replay is provably sound.

Parallelism the reference does not have. Reference sketching chunked, index storage sharded by hash, FASTA parsing a two-pass count-then-fill — all parallel, none order-dependent, so the index does not depend on how many threads built it and output stays byte-identical.

Hot-loop work. Repetitive seeds accumulate in place at constant integer cost rather than through a scratch hash map; the rolling nucleotide hash [3] keeps rotations precomputed and iteration free of bounds checks; and the original's packed-key bucket sort is made deterministic, which the reference's `std::sort` does not guarantee.

Exactness is enforced, not asserted: output is identical across every thread count (15 of 15 checks) and against the reference, and any drop in mapped or confidently-mapped reads blocks a merge.

3 Results

Commit `cc90fa548767`, rustc 1.97.1, suite 1.0: 5 benchmarks × 3 metrics, 15 cells behind every range, at $k = 25$, $r = 0.01$, $\theta = 0.4$, $\delta = 0.075$, $\phi = 0.3$. Machines: `a2` (x86_64, 4 sockets × 16 cores, 4 NUMA nodes, 64 cores) and `galaxy` (aarch64, 1 socket × 128 cores, 1 NUMA node, 128 cores), measured 2026-08-10 and 2026-08-10.

Both machines run the same computation, which licenses every comparison below. The mapper is deterministic, so reads mapped, confident calls and buckets seeded, refined and reported must be identical on both for one commit. They are, on 15 of 15 pairs.

Speed. Table 1, both terms of each ratio from the same host so host speed cancels: 2.14–2.97× (median 2.62×) on `a2`, 1.94–2.59× (median 2.29×) on `galaxy`. The reference is built `-O3 -march=native -flto`, so this is no naive baseline.

Memory, the larger result. 2.58–2.67 GB single-threaded against 18.85 GB, a 7.0–7.3× reduction — but what changes with the ratio matters more. The reference sizes its accumulator against the genome, so 18.85 GB is what it costs to map *one* read and what it costs to map millions: a property of the index, which a machine either has room for or cannot run the tool at all. Sizing by the read's half-length makes it a property of the query. Figure 1 is why we call the headline

Benchmark	Metric	x86_64	aarch64	Δ
B01	Containment	2.66×	2.39×	-0.28
B01	Jaccard	2.34×	2.26×	-0.07
B01	bucket_SH	2.88×	2.52×	-0.36
B02	Containment	2.76×	2.48×	-0.28
B02	Jaccard	2.61×	2.29×	-0.31
B02	bucket_SH	2.97×	2.59×	-0.38
B03	Containment	2.55×	2.16×	-0.39
B03	Jaccard	2.48×	2.10×	-0.37
B03	bucket_SH	2.62×	2.22×	-0.40
B04	Containment	2.22×	1.95×	-0.27
B04	Jaccard	2.14×	1.94×	-0.20
B04	bucket_SH	2.19×	1.98×	-0.21
B05	Containment	2.84×	2.51×	-0.32
B05	Jaccard	2.74×	2.37×	-0.37
B05	bucket_SH	2.89×	2.59×	-0.30

The C++ rows for x86_64 (2026-08-10), aarch64 (2026-08-10) were carried forward from an earlier run, so that column divides two measurement days and its cancellation of machine speed is only approximate.

Table 1: shmap-rs against the C++ shmap, measured separately on each machine. Unlike every other timing table here, this one is not confounded by the hosts differing: both terms of each ratio come from the same machine, so its speed cancels and what remains is a property of the two programs.

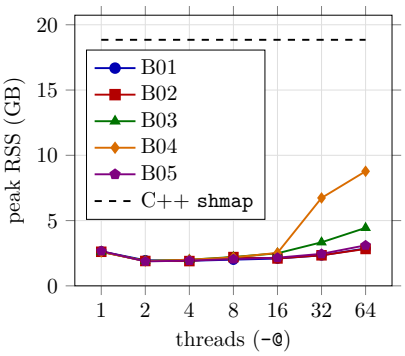

Figure 1: Peak resident memory against thread count, Containment, with the C++ reference as a horizontal line.

single-threaded: each worker carries its own accumulator, so peak RSS bottoms at 1.85 GB at 2 threads — under the single-threaded figure, where bounded read-ahead dominates — and reaches 8.78 GB at 64 on B04. Even that worst cell keeps a 2.1× advantage, on cores the reference cannot use.

Where the time goes. Figure 2 gives every stage's share of query_mapping for all 15 cells. The costliest is map: match_rest at 21.0% of CPU on average: pointer-chasing and latency-bound, which is why the profitable changes concerned the shape of the data rather than the width of the arithmetic. The profile barely moves between machines — largest shift in any stage's share 6.5 percentage points, in index: reading, which is input reading.

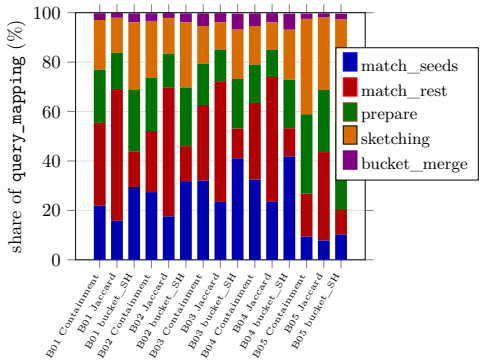

Figure 2: Where mapping time goes: stage shares of query_mapping, single-threaded.

Thread scaling, and where it stops. On B04 a2 reaches 16.6× at 32 threads then falls back; galaxy climbs monotonically to 27.3× at 64. Matrix medians 6.2× against 10.1× at 16 threads, 6.0× against 12.0× at 64. Not the instruction set but the topologies above: the ceiling is memory-bandwidth contention on the shared index, appearing as soon as a run spans more than one socket; numactl to one node recovers most of it.

Accuracy is unchanged, and checked. On the simulated benchmark, the only input with ground truth, reads land within one read length of their true position for 98.76–99.21% of reads by metric, and at most 6 reads under any metric are placed wrongly while reported as confident.

Four things that did not pay, recorded because a negative result nobody writes down gets retried. The sketching loop is load-port bound at six level-one loads per base, which sinks both SIMD (more lanes, same ports) and two-bit packing (removes the wrong two loads). Prefetching the pruning lookups is at best break-even. Per-node index replication, in five implementations, was slower than doing nothing wherever a run spanned more than one socket.

4 Conclusion

The headroom was not algorithmic — which is the point. Ivanov's seed heuristic on sketches is what makes the mapping decisions, and it is unchanged here down to the byte; the change that mattered most was a data structure sized by the query rather than by the reference. Every number, table and figure here is regenerated from the committed result sets by scripts in the repository, and CI fails if any drifts from what it reports.

Acknowledgements. Dobromir Petrov contributed to this work as a project member.

References

[1] Ivanov,P. and Medvedev,P. map-shmap: practical long-read mapping with a seed heuristic on sketches. Manuscript in preparation.

[2] Ondov,B.D. *et al.* (2016) Mash: fast genome and metagenome distance estimation using MinHash. *Genome Biol.*, 17, 132.

[3] Mohamadi,H. *et al.* (2016) ntHash: recursive nucleotide hashing. *Bioinformatics*, 32, 3492–3494.